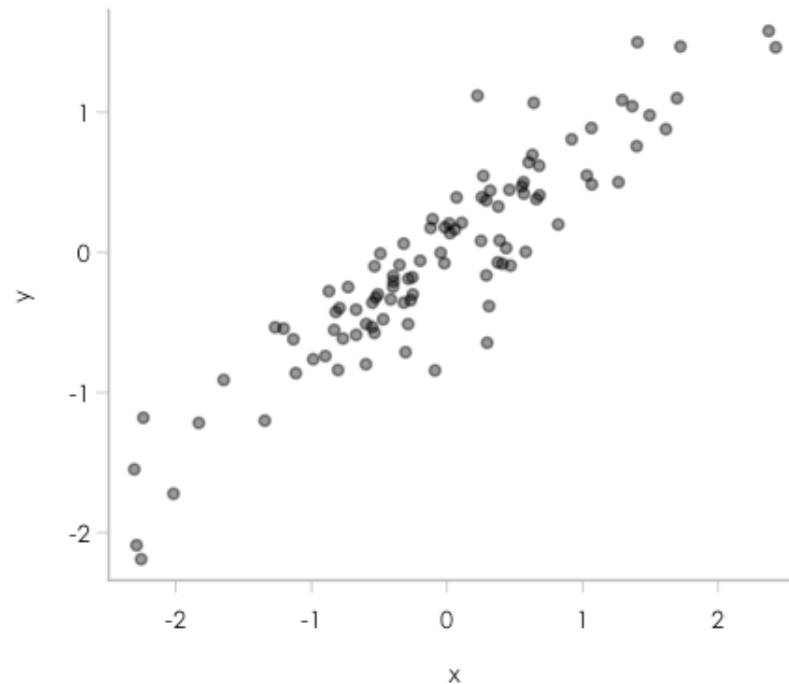
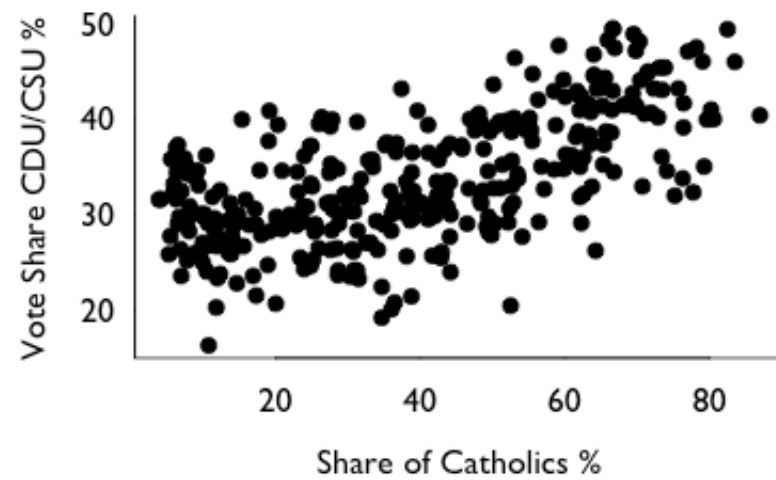


Correlation, Correlation

## The Scatterplot



“Indeed, among all the forms of statistical graphics, the humble scatterplot may be considered the most versatile, polymorphic, and generally useful invention in the entire history of statistical graphics.” (Friendly & Denis 2005)



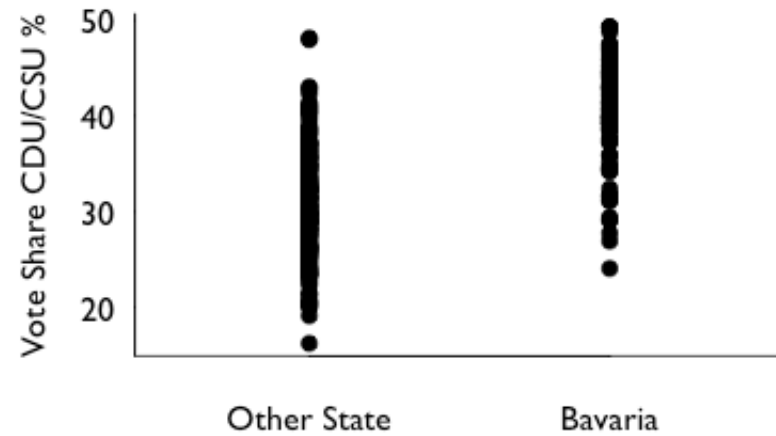
Correlation?

Functional Form?

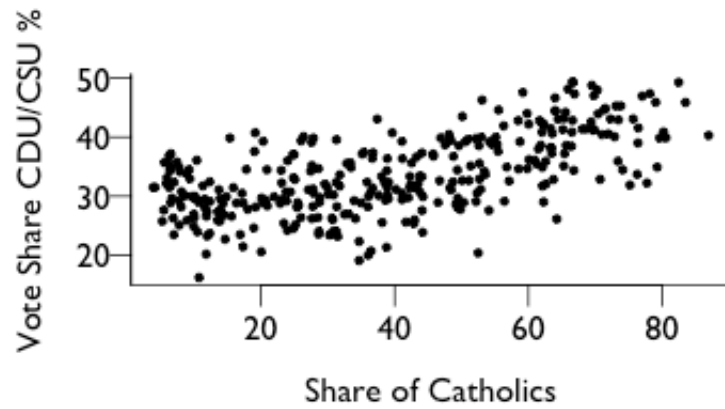
Clusters?

Gaps?

Outliers?

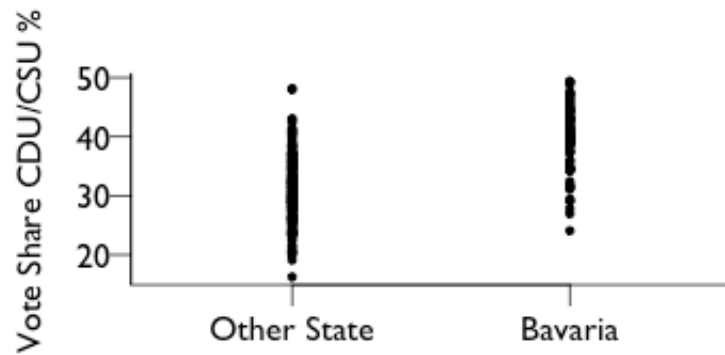


## Reducing Overplotting

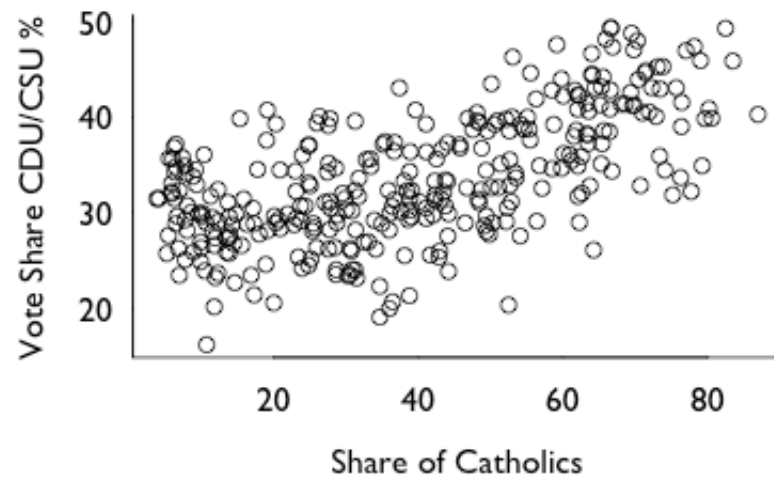


Decrease symbol size.

`cex = .5`

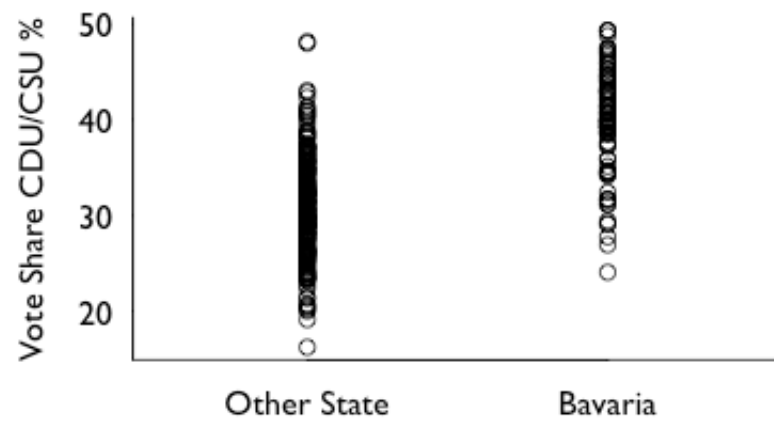


## Reducing Overplotting



Remove fill color.

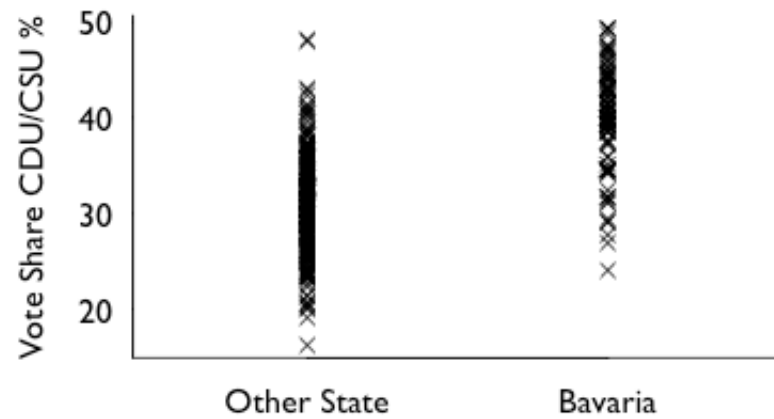
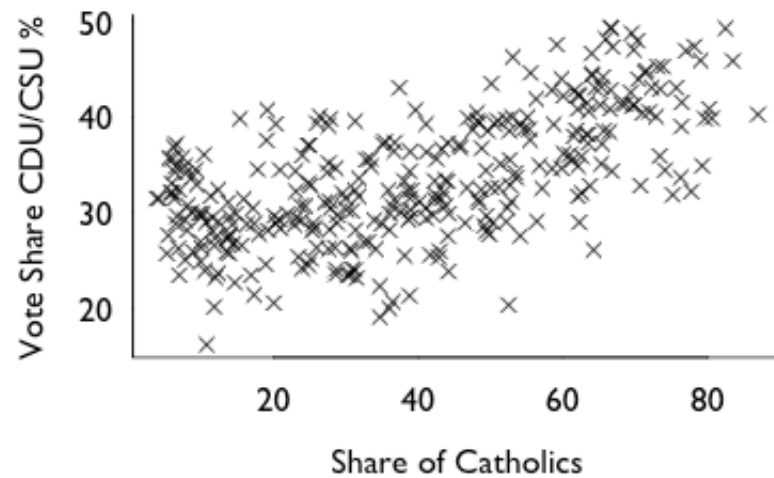
`pch=21`



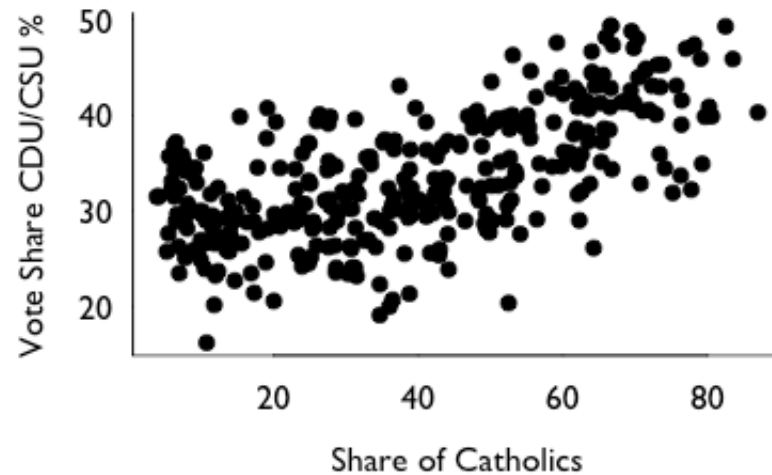
## Reducing Overplotting

Choose other symbols.

`pch=4`



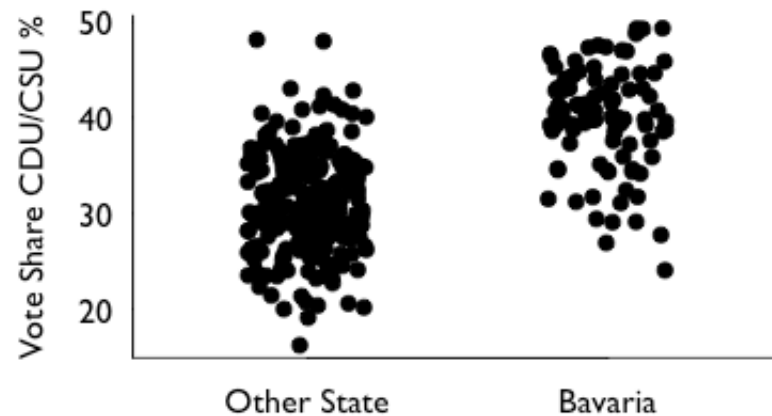
## Reducing Overplotting



Jittering.

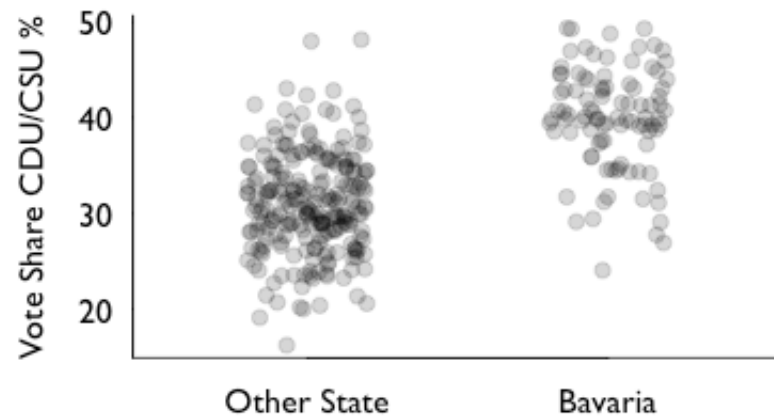
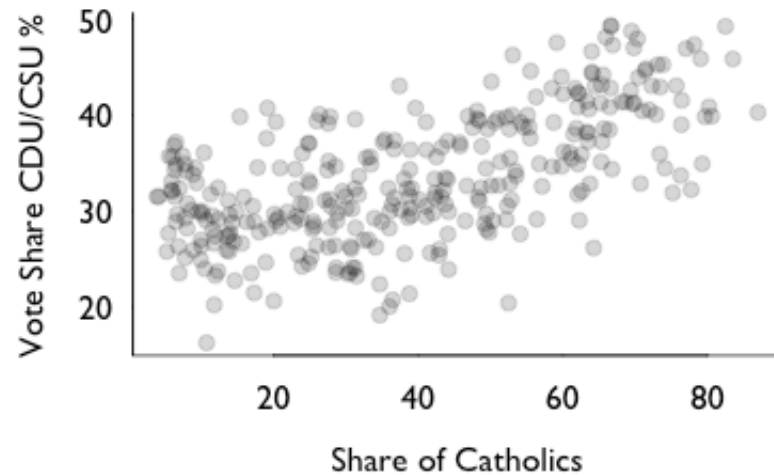
Adds random noise to data values.

Doesn't make sense  
for continuous variables!



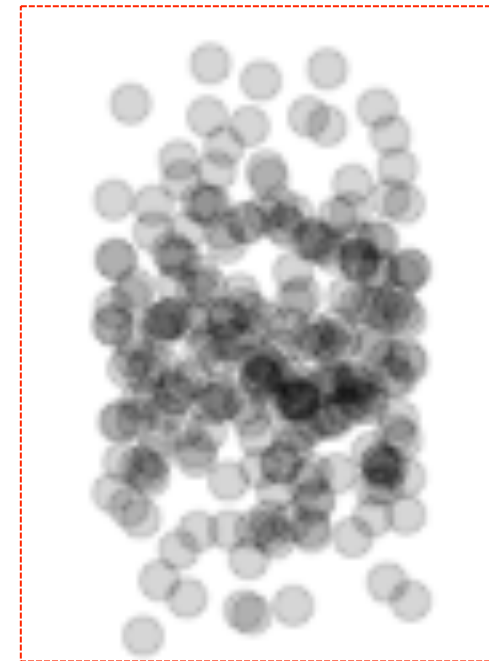
```
points(jitter(daten05$bayern),  
       daten05$cdu, pch=19)
```

## Reducing Overplotting



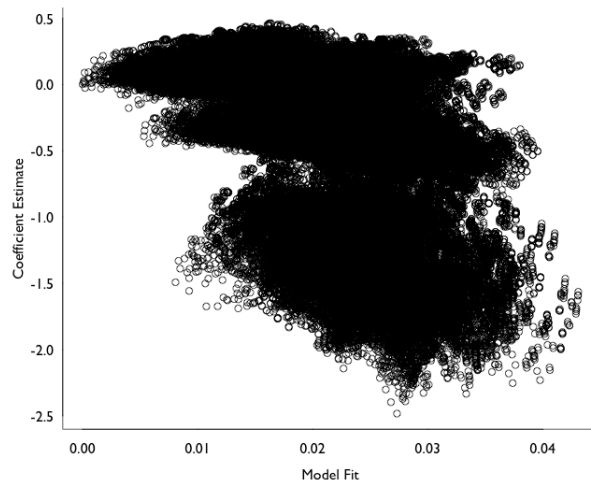
## Alpha Blending.

Makes plotting symbols transparent.

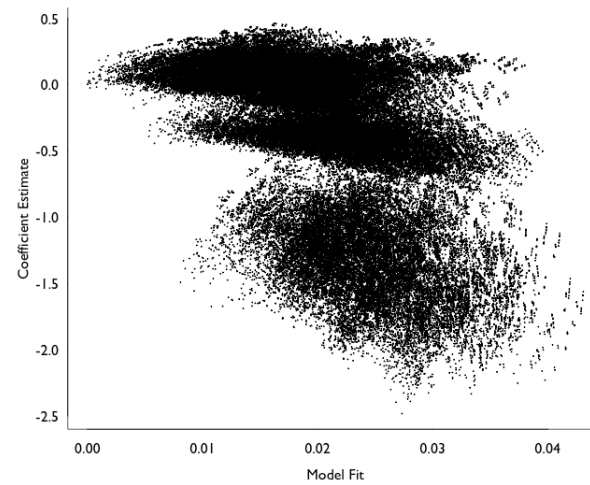


```
points(jitter(daten05$bayern),  
daten05$cdu, pch=19, col=rgb(00,  
00, 00, 50, max=255))
```

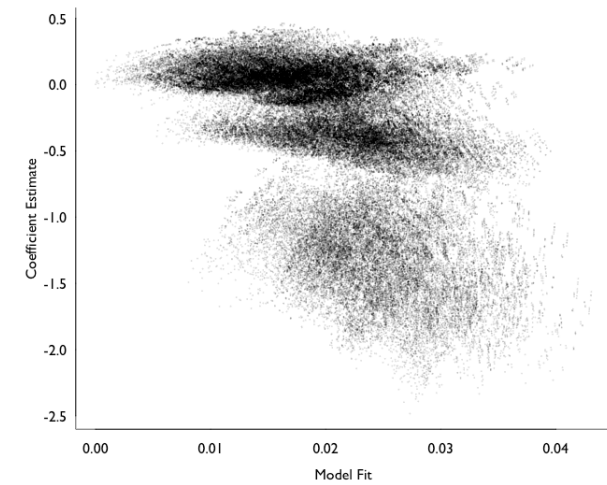
## Same with „Big Data“ Example



No fill color...



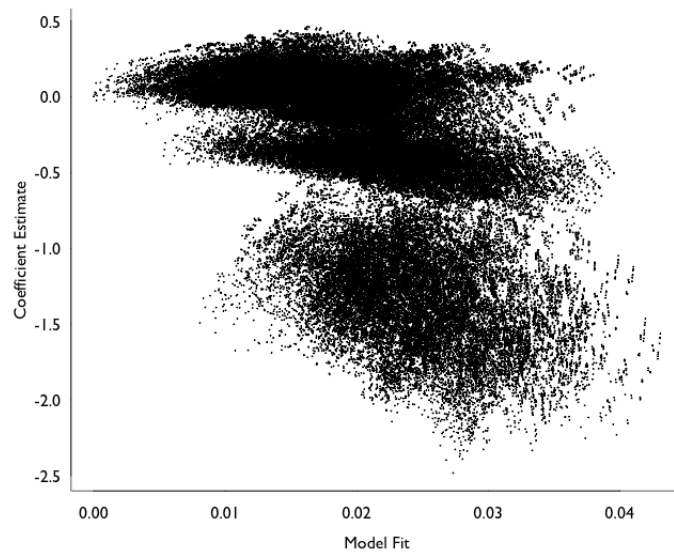
Smaller symbols...



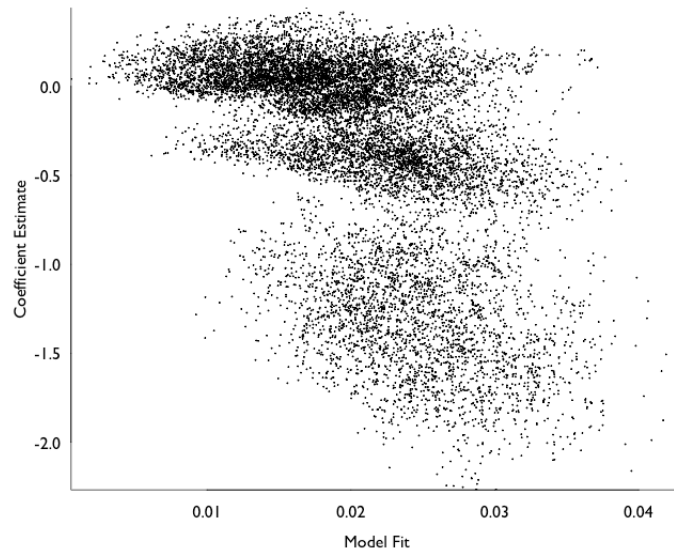
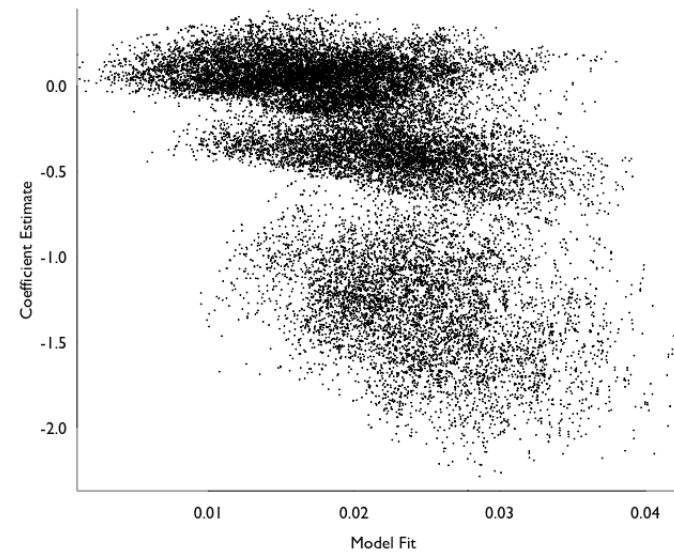
Small symbols + Alpha blending

# Sampling

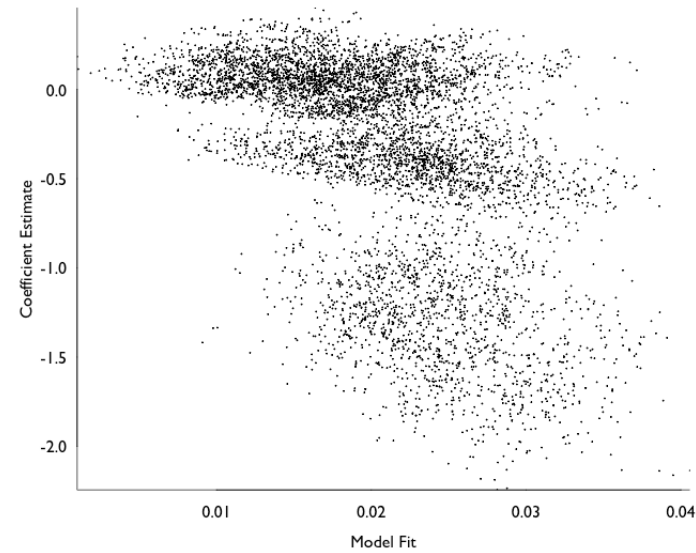
262143 Points



20% Sample



10% Sample

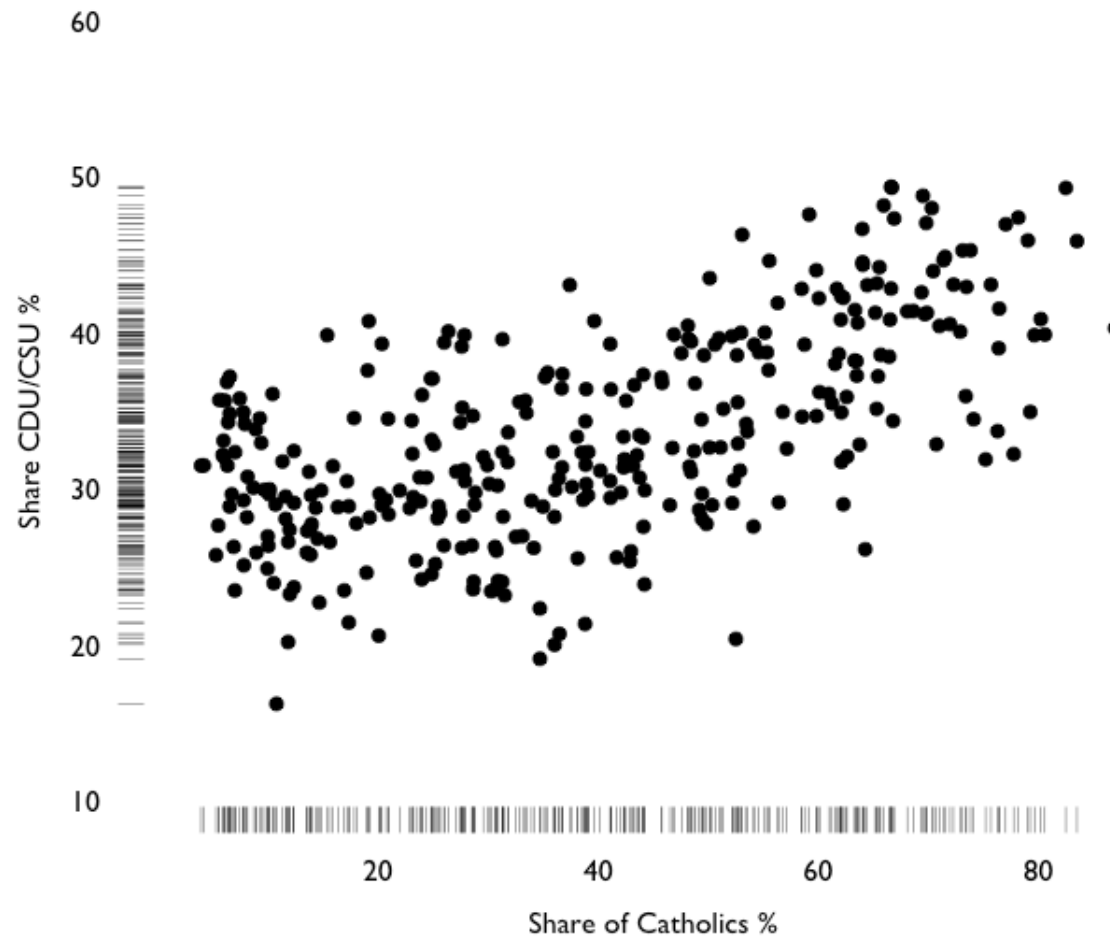


5% Sample

## Sampling

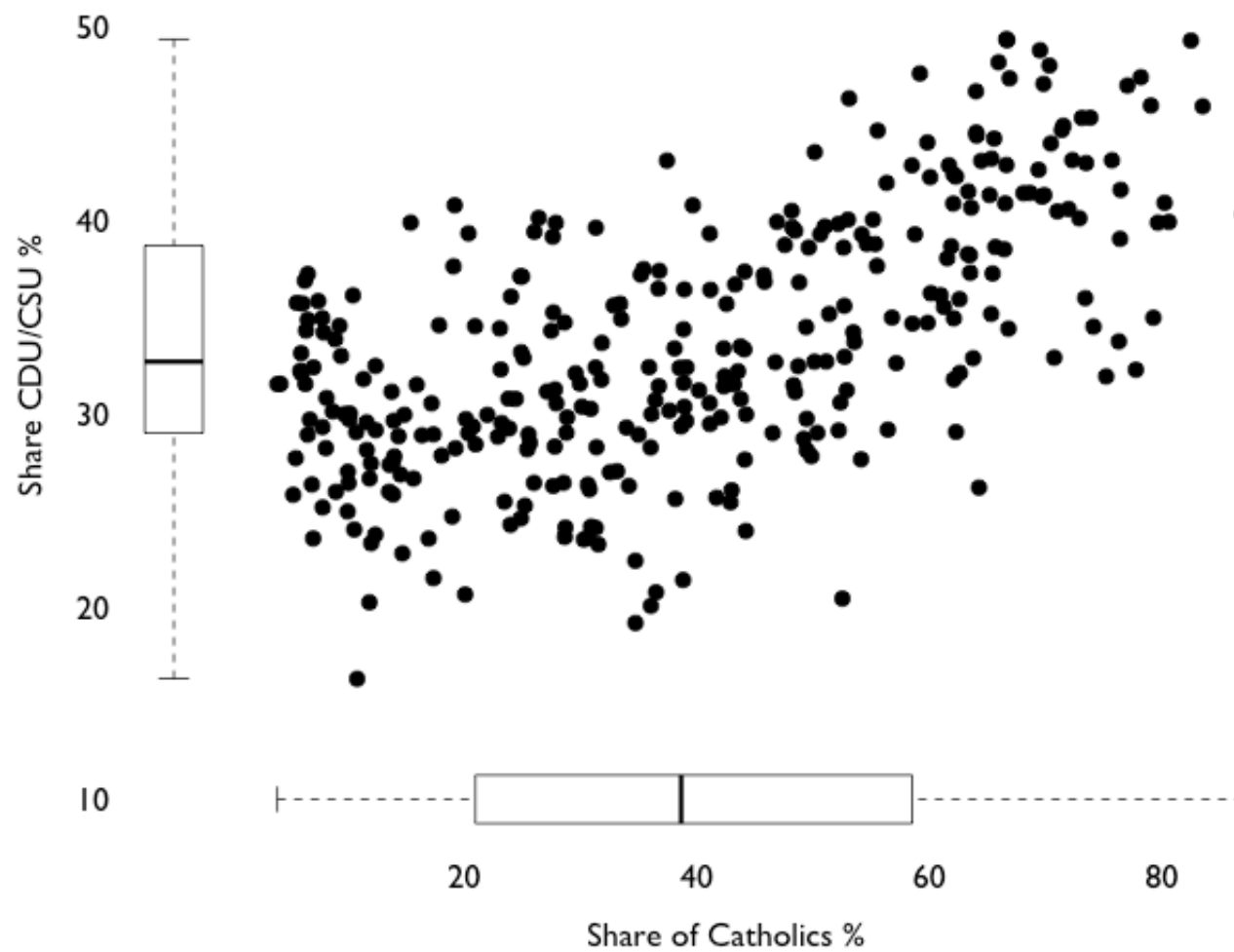
```
samp.20 <- sample(length(DATA$Est.z[DATA$variable=="milper"]),  
round(length(DATA$Est.z[DATA$variable=="milper"])*.2))  
  
plot(DATA$dev_diff[DATA$variable=="milper"][samp.20],  
      DATA$Est.z[DATA$variable=="milper"][samp.20], pch=19, cex=.1,  
axes=F, xlab="Model Fit", ylab="Coefficient Estimate")  
axis(1)  
axis(2, las=1)  
box(bty="l")
```

## Enhancing Scatterplots: Rugs



```
rug(daten05$kath, side=1)  
rug(daten05$cdu, side=2)
```

## Enhancing Scatterplots: Boxplots



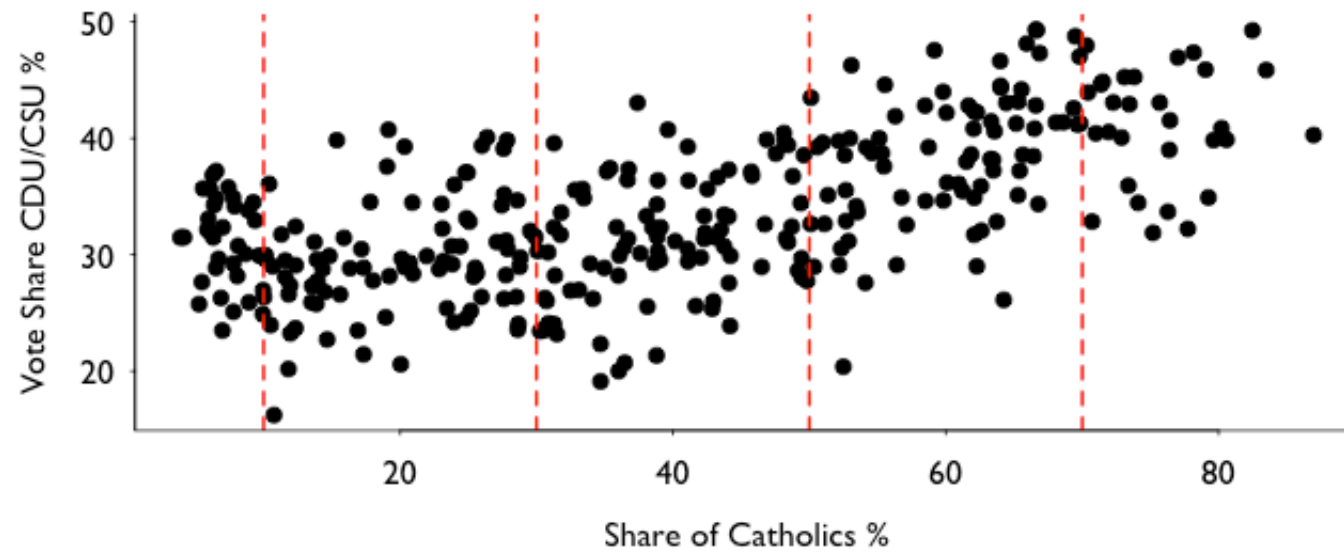
## Enhancing Scatterplots: Boxplots

```
plot(daten05$kath, daten05$cdu, pch=19, axes=F, cex.lab=1, tck=-.01,  
     main="", xlab="Share of Catholics %", ylab="Share CDU/CSU %", las=1,  
     ylim=c(.1, .55), xlim=c(-.05, .9))  
axis(2, at=seq(.10, .60, by=.10), labels=c("10", "20", "30", "40", "50",  
     "60"), las=1, col="white")  
axis(1, at=seq(.20, .80, by=.20), labels=c("20", "40", "60", "80"),  
     las=1, col="white")
```

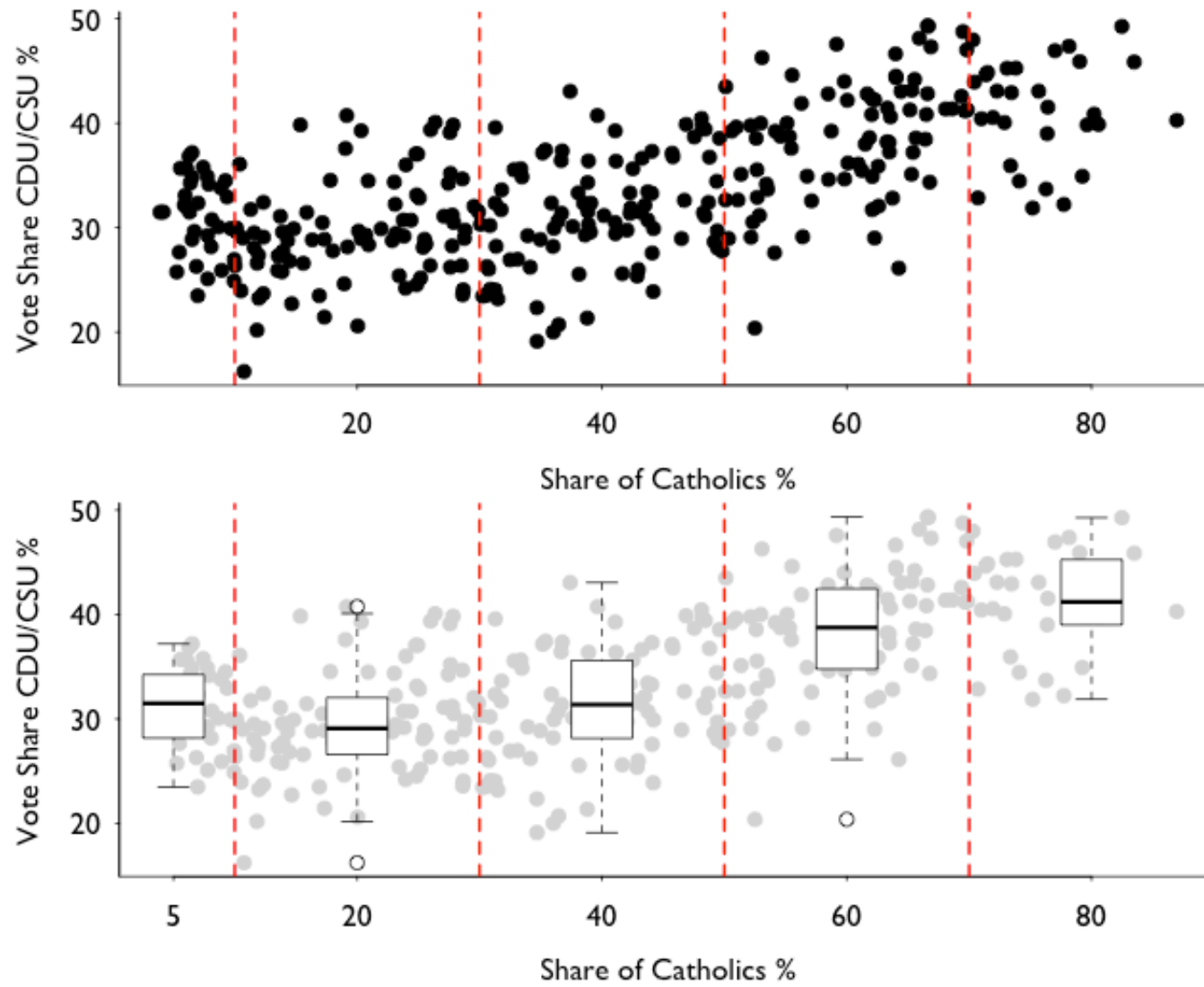
```
boxplot(daten05$kath, at=.1, axes=F, horizontal=T, add=T, boxwex=.05)
```

```
boxplot(daten05$cdu, at=-.05, axes=F, horizontal=F, add=T, boxwex=.10)
```

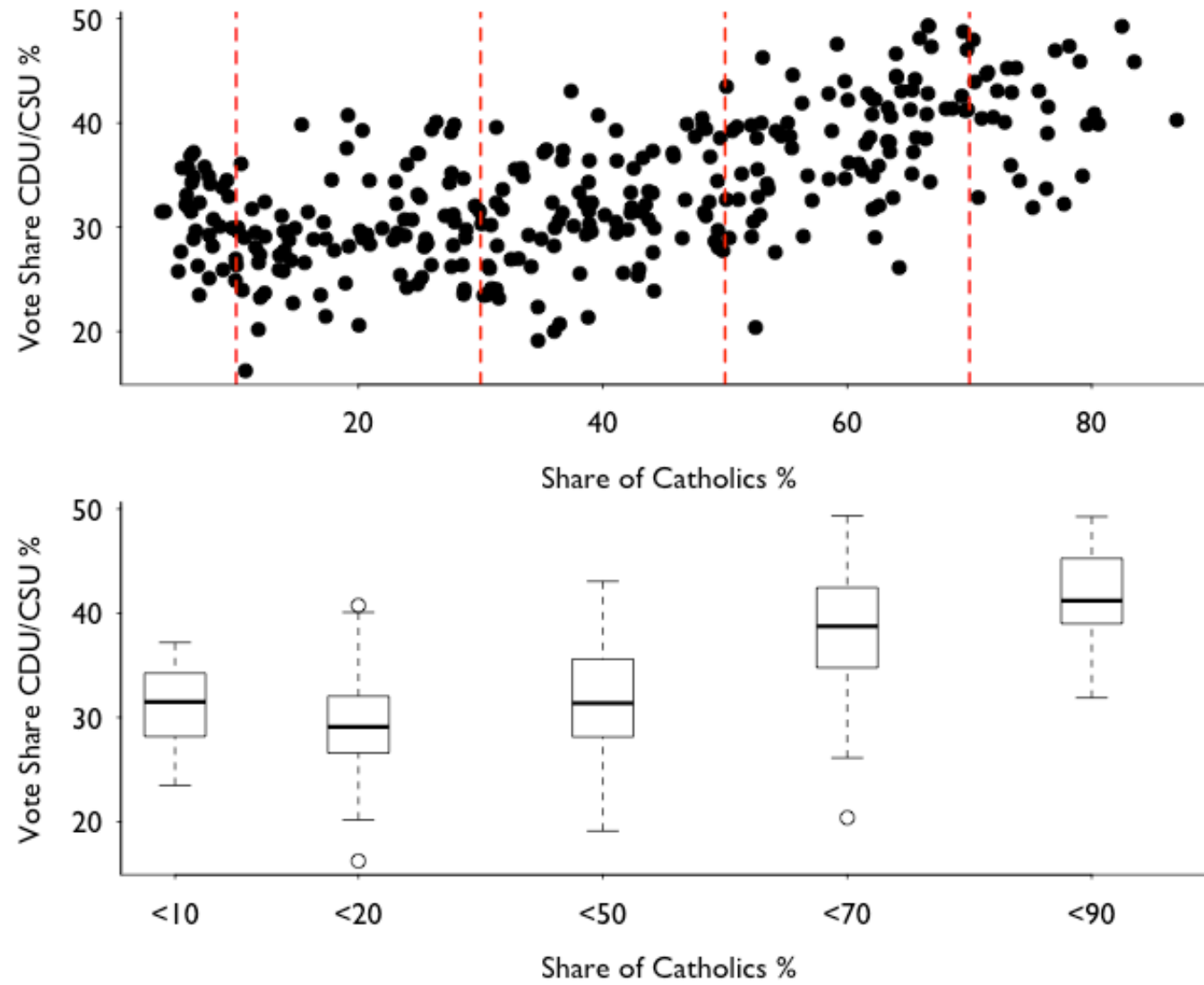
## Slicing



## Slicing



## Slicing



## Fitting

“There are two components to visualizing the structure of statistical data - [graphing](#) and [fitting](#).

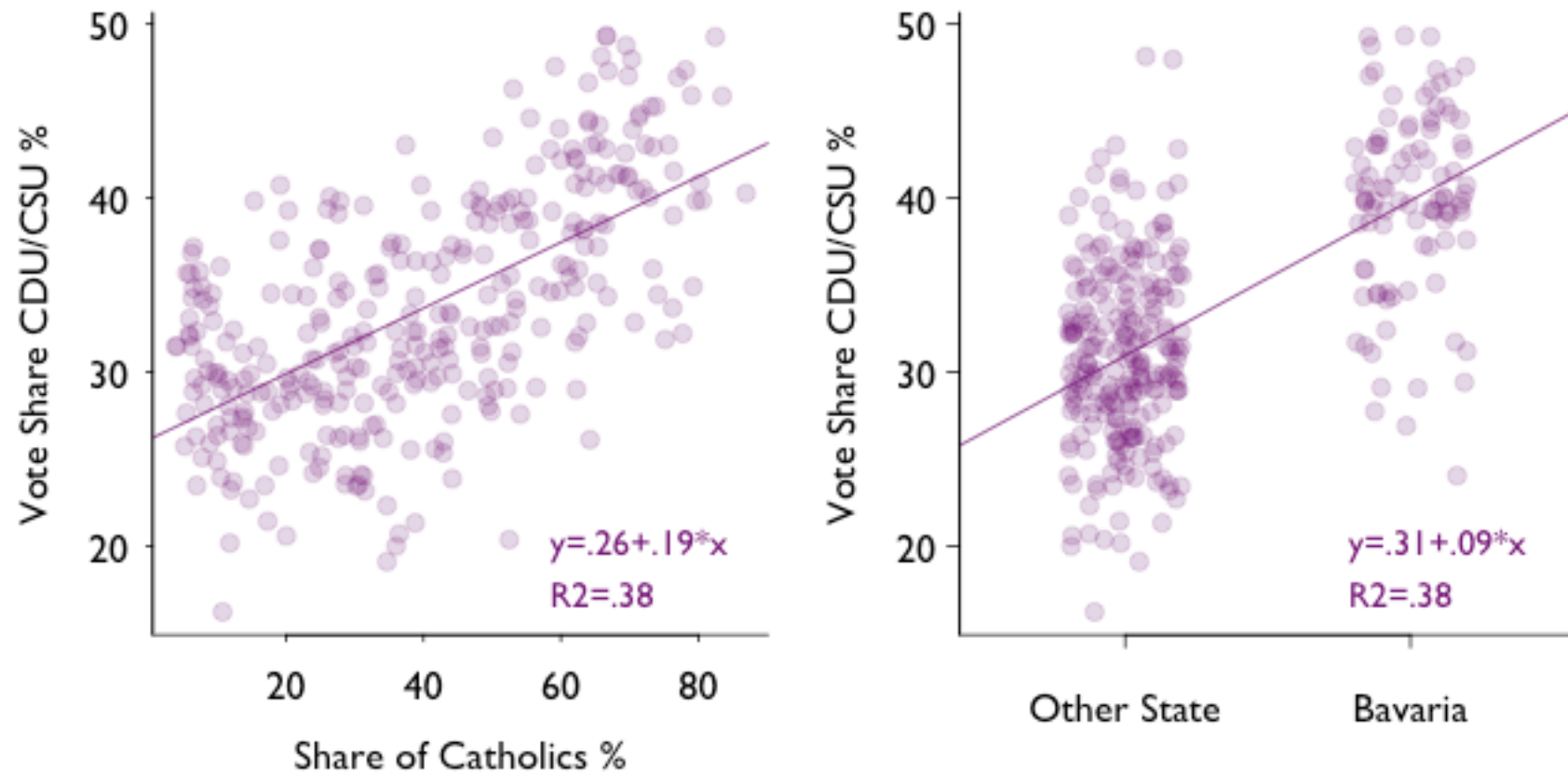
Graphs are needed, of course, because visualization implies a process in which information is encoded on visual displays.

Fitting mathematical functions to data is needed too.

Just graphing raw data, without fitting them and without graphing the fits and residuals, often leaves important aspects of data undiscovered”

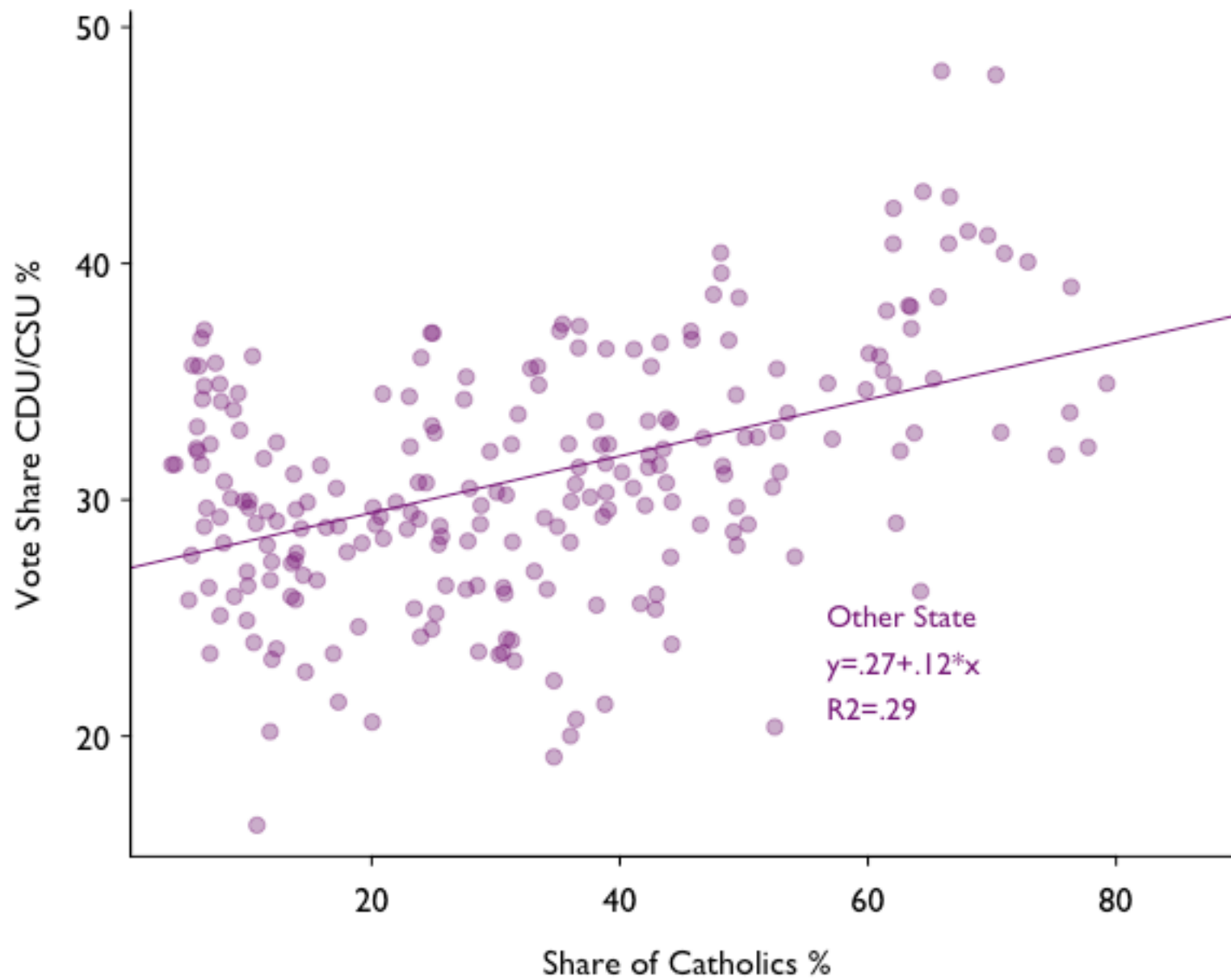
(Cleveland 1993).

## Parametric Fitting

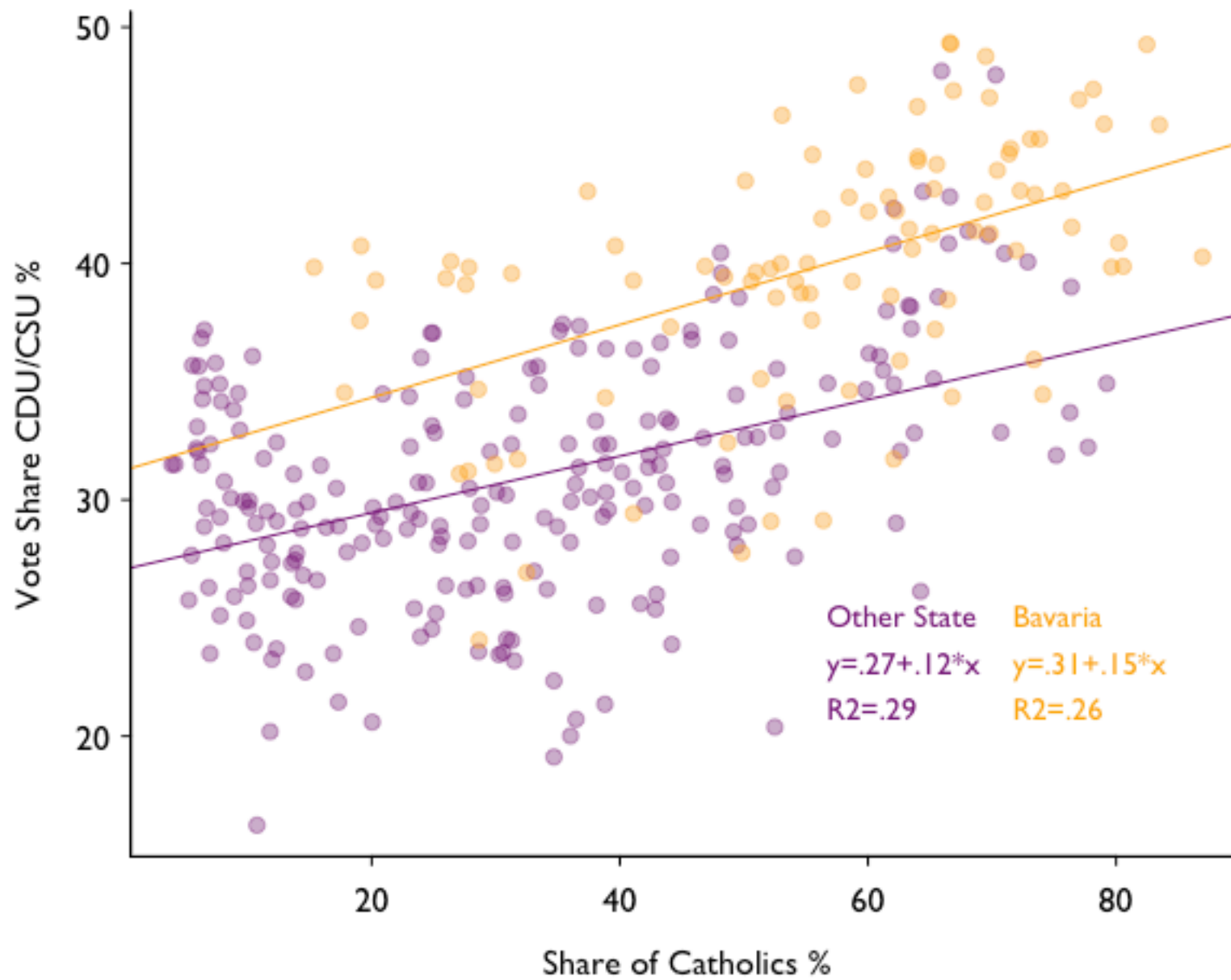


```
abline(lm(daten05$cdu~daten05$kath), col=rgb(.47, 00, .47))  
text(.55, .20, "y=.26+.19*x", col=rgb(.47, 00, .47), pos=4, cex=.9)  
text(.55, .17, "R2=.38", col=rgb(.47, 00, .47), pos=4, cex=.9)
```

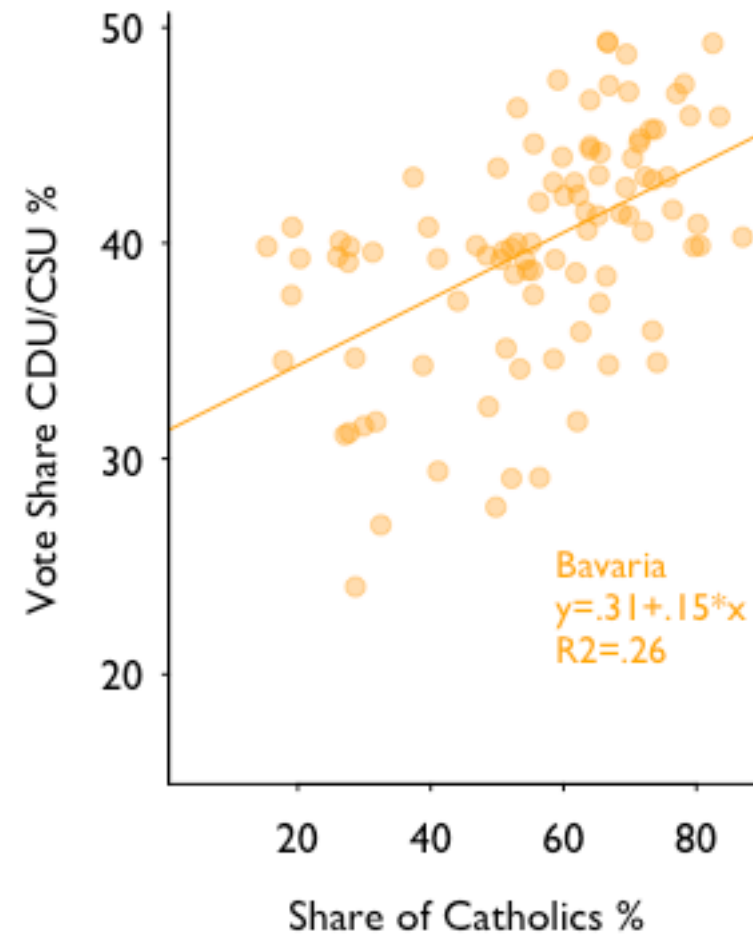
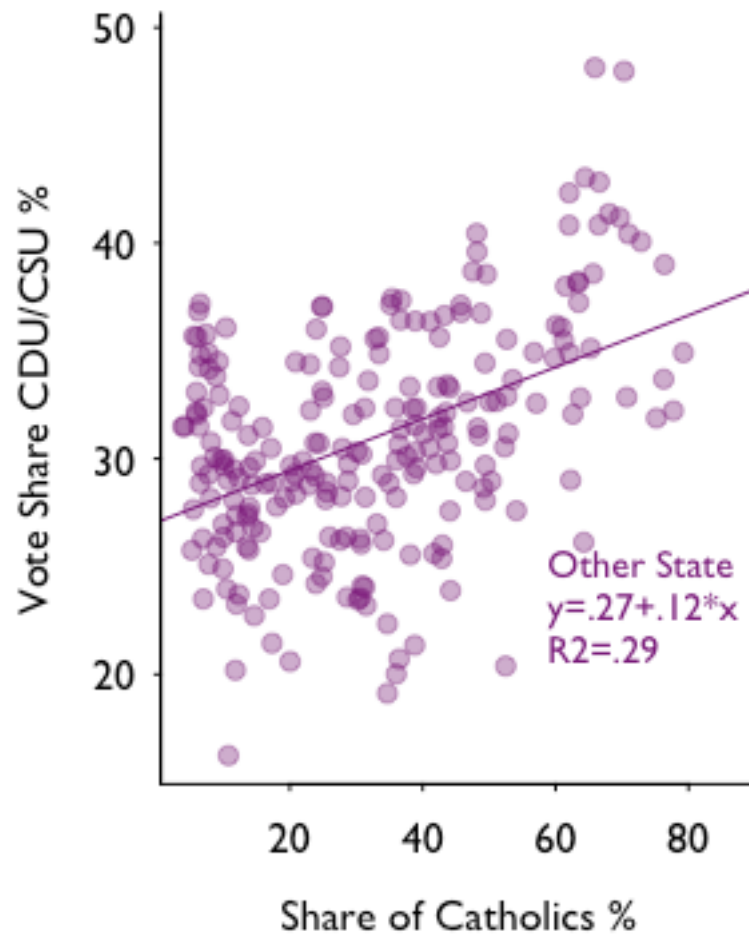
## Parametric Fitting



## Parametric Fitting

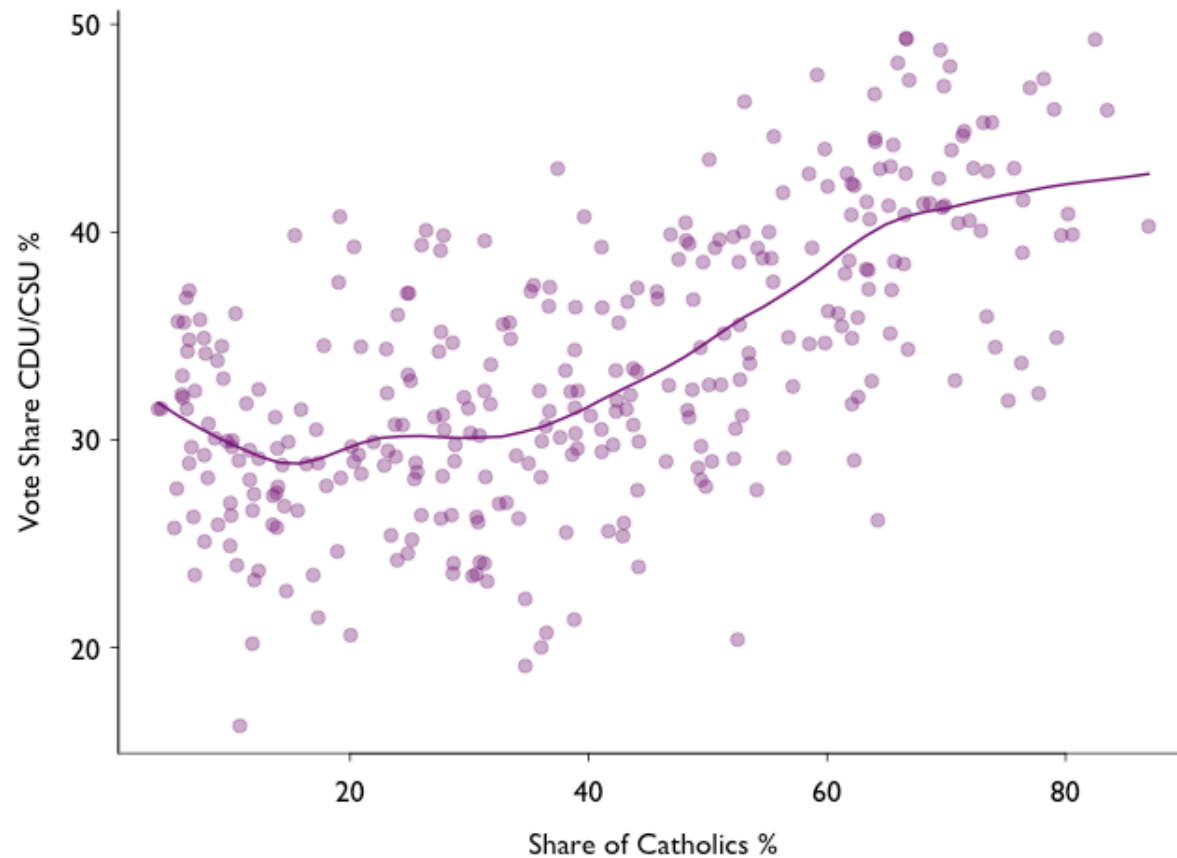


## Parametric Fitting



# Non-Parametric Fitting: Scatter Plot Smoothers

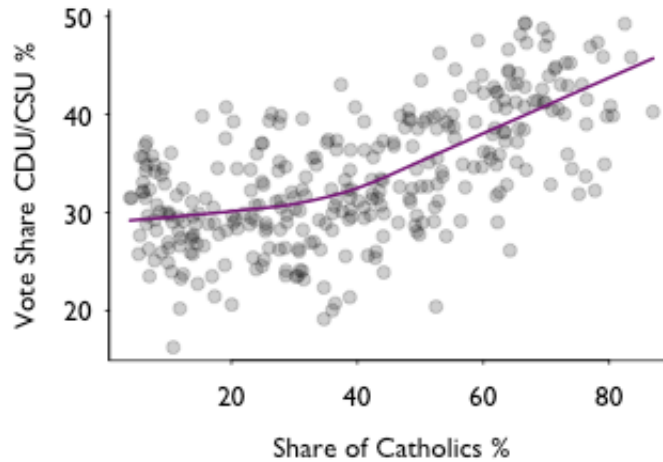
*LOESS: locally weighted regression*



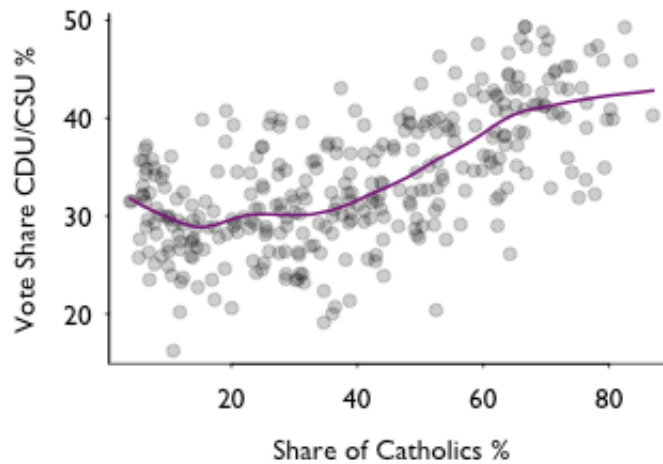
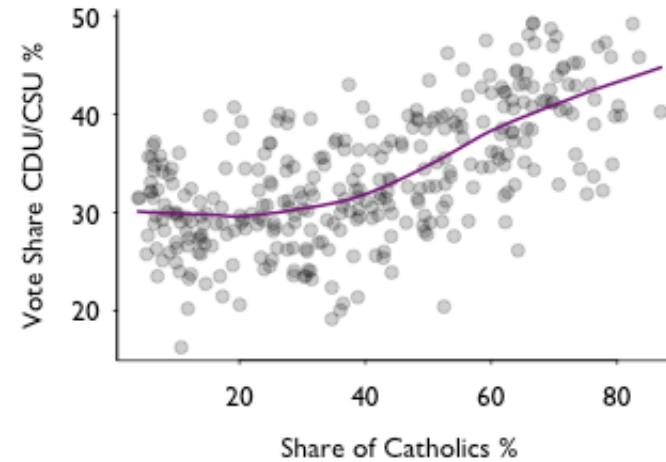
```
lines(loess.smooth(daten05$kath, daten05$cdu, span=.3, degree=1),  
col=rgb(.47, 00, .47), lwd=2)
```

# Non-Parametric Fitting: Scatter Plot Smoothers

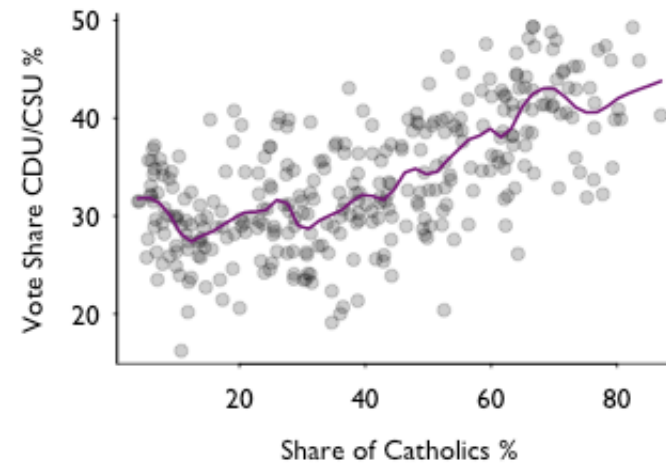
span=.8



span=.5

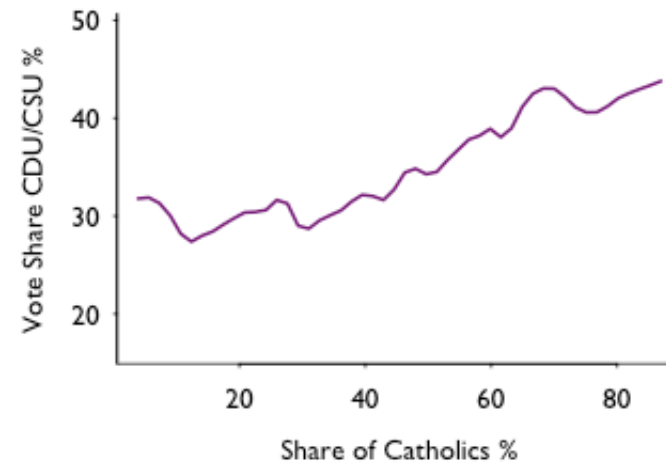
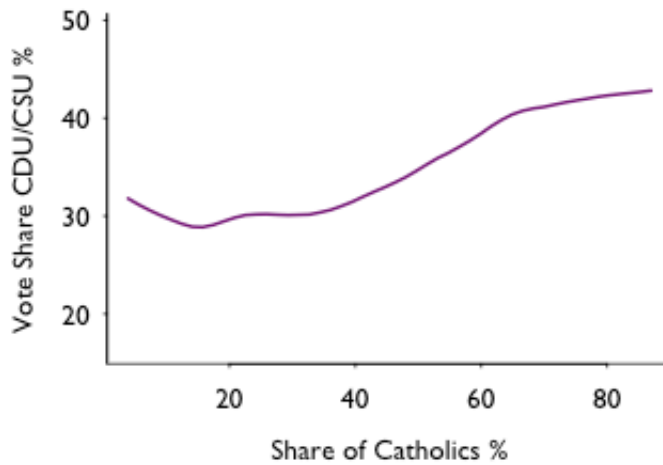
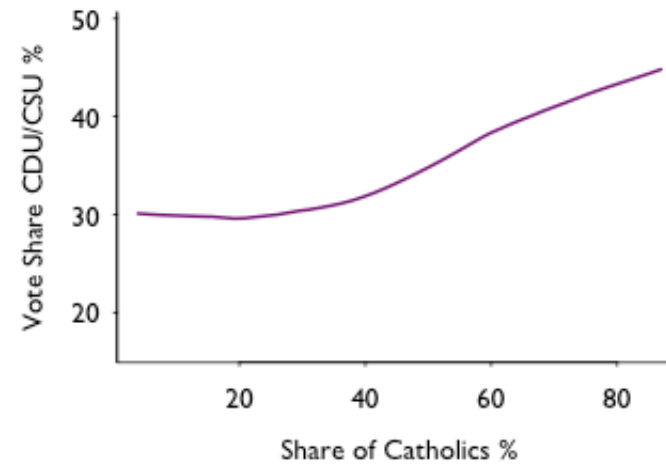
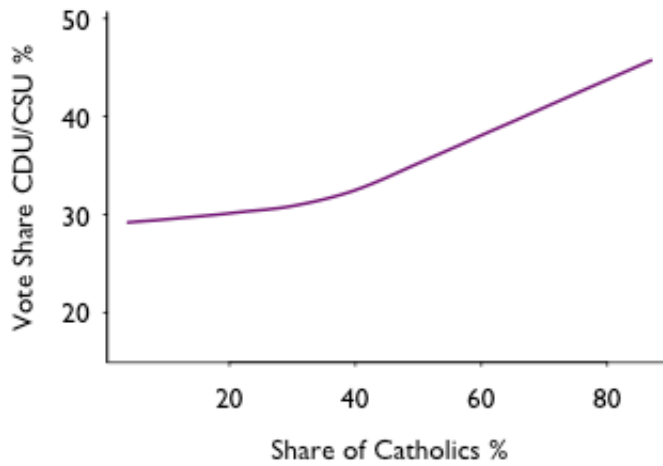


span=.3



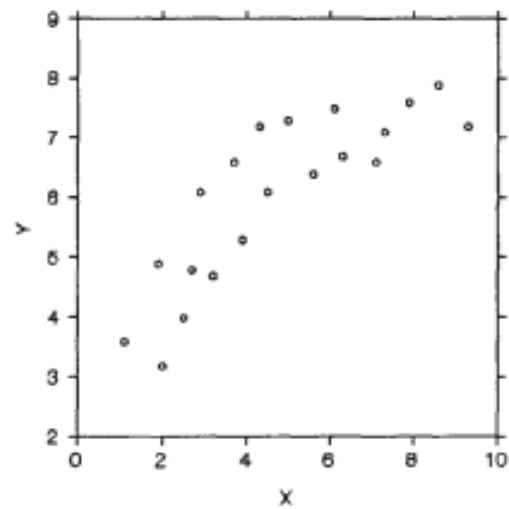
span=.1

## Non-Parametric Fitting: Scatter Plot Smoothers

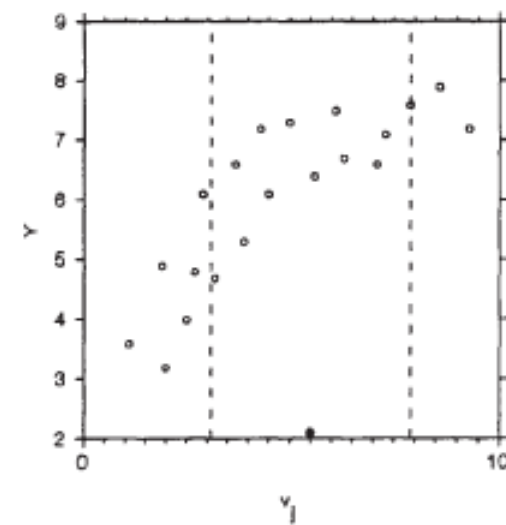




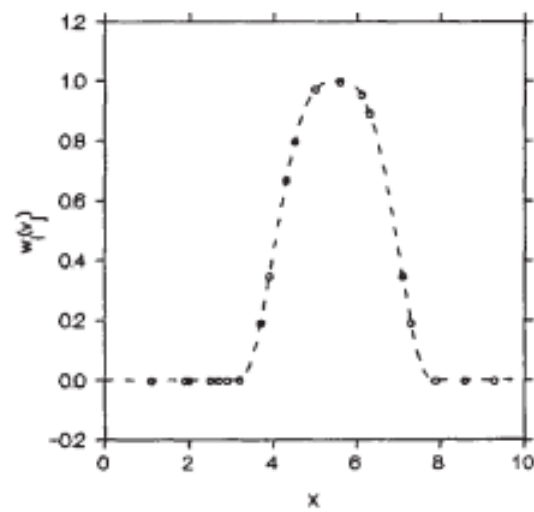
A. Hypothetical Data for Loess Fit.



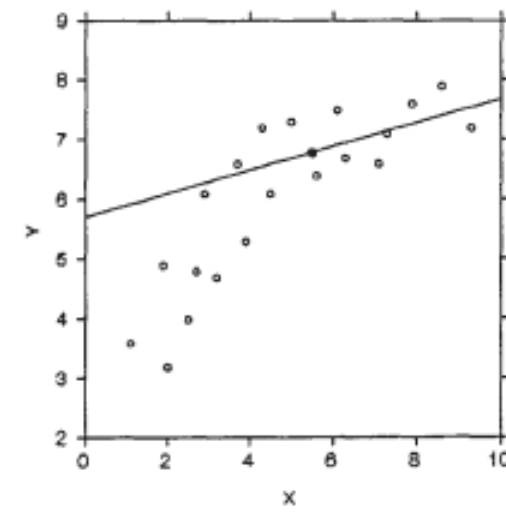
B. Window for  $v_j=5.5$  and  $\alpha=0.6$ .



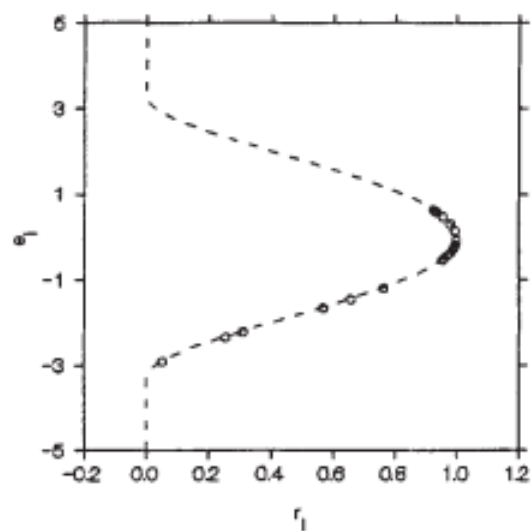
C. Tricube Neighborhood Weights.



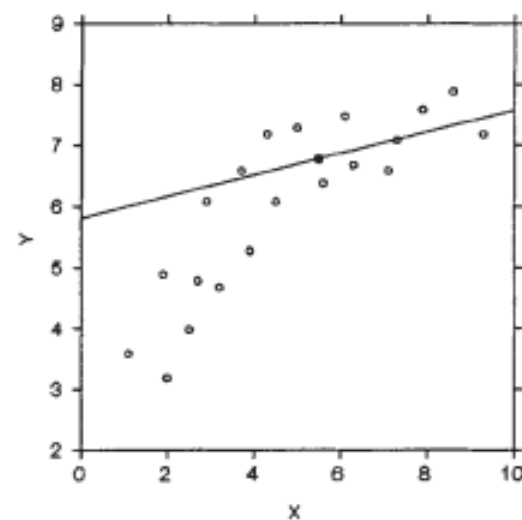
D. Initial Regression Line and Fitted Value.



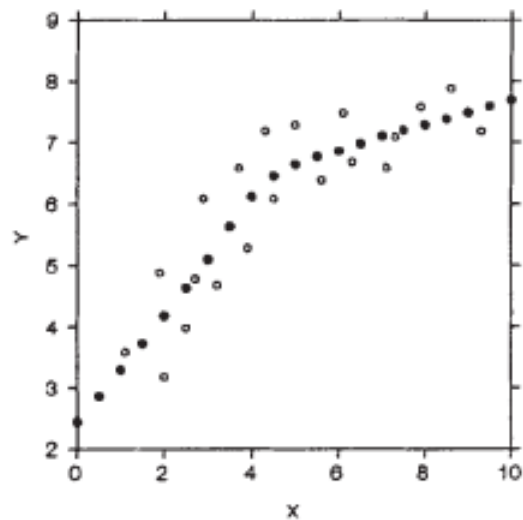
E. Bisquare Robust Weights.



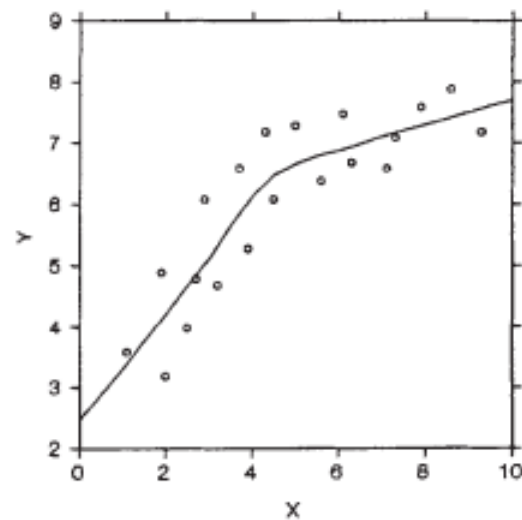
F. Final Robust Line and Fitted Value.



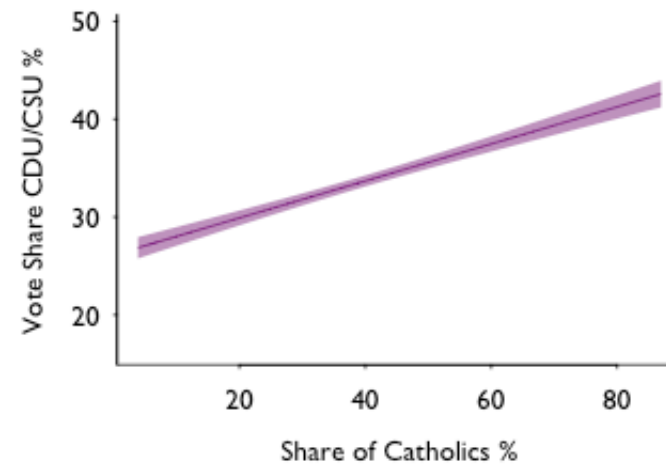
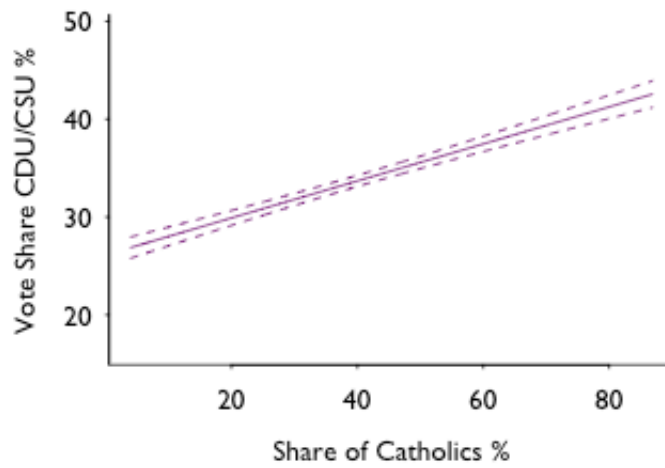
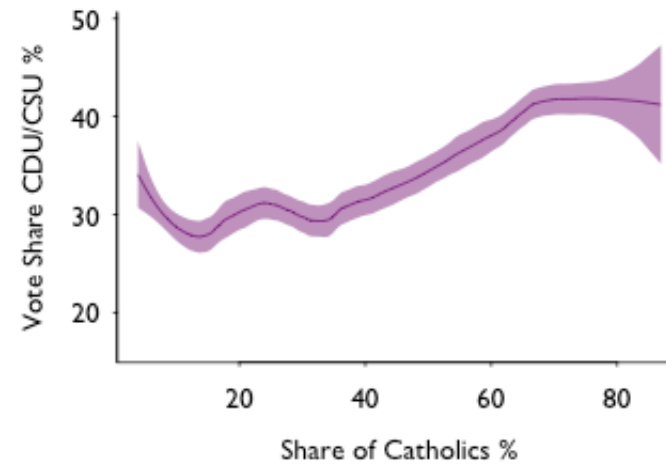
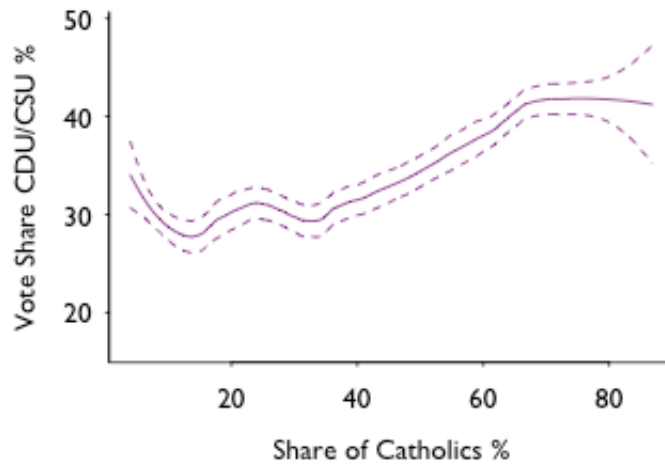
G. Complete Set of Fitted Values.



H. Loess Curve



## Add Confidence Intervals



# Add Confidence Intervals

## Extract Fitted Values and SEs from loess

```
l.p <- predict(loess(daten05$cdu ~ daten05$kath, span=.3), se=T)
l.p <- cbind(daten05$kath, l.p$fit, l.p$fit - qt(0.975,
  l.p$df)*l.p$se, l.p$fit + qt(0.975, l.p$df)*l.p$se)
l.p <- l.p[order(l.p[,1]),]
```

## Plot as Lines

```
lines(l.p[,1], l.p[,2], col=rgb(.47, 00, .47))
lines(l.p[,1], l.p[,3], col=rgb(.47, 00, .47), lty=2)
lines(l.p[,1], l.p[,4], col=rgb(.47, 00, .47), lty=2)
```

## Plot as Polygon

```
lines(l.p[,1], l.p[,2], col=rgb(.47, 00, .47))
polygon(c(l.p[,1], rev(l.p[,1])), c(l.p[,3], rev(l.p[,4])),
  col=rgb(.47, 00, .47, .50), border=F)
```

# Add Confidence Intervals

## Extract Fitted Values and Confidence Intervals from OLS

```
ols <- lm(daten05$cdu~daten05$kath)
r.p <- predict(ols, interval="confidence")
r.p <- cbind(daten05$kath, r.p)
r.p <- r.p[order(r.p[,1]),]
```

## Plot as Lines

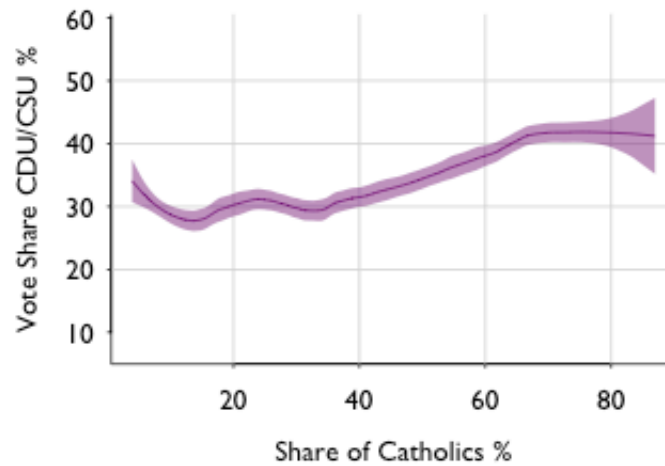
```
lines(r.p[,1], r.p[,2], col=rgb(.47, 00, .47))
lines(r.p[,1], r.p[,3], col=rgb(.47, 00, .47), lty=2)
lines(r.p[,1], r.p[,4], col=rgb(.47, 00, .47), lty=2)
```

## Plot as Polygon

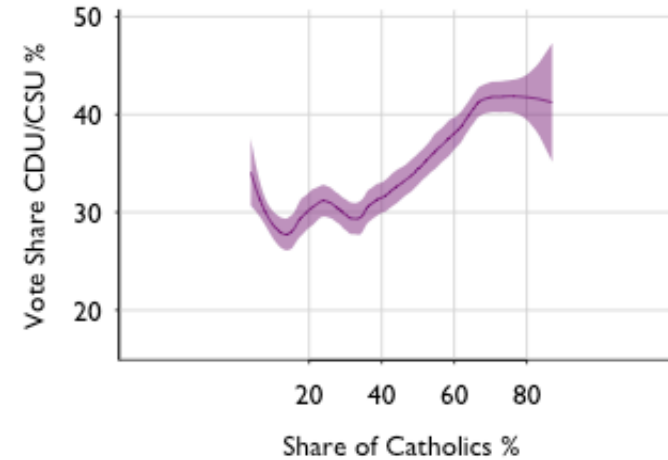
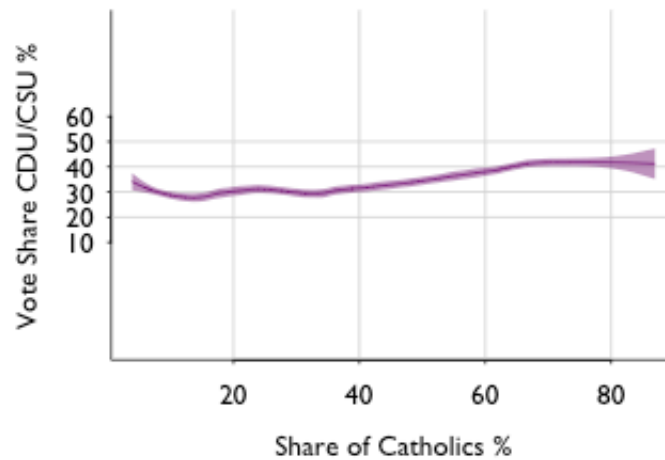
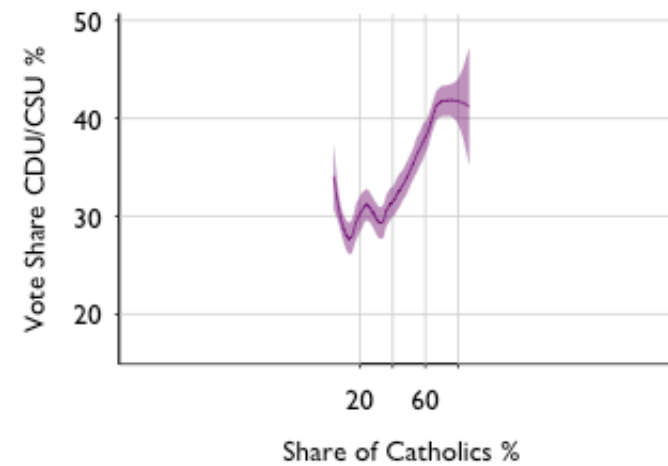
```
lines(r.p[,1], r.p[,2], col=rgb(.47, 00, .47))
polygon(c(r.p[,1], rev(r.p[,1])), c(r.p[,3], rev(r.p[,4])),
  col=rgb(.47, 00, .47, .50), border=F)
```

Aspect Ratio = Height/Width of Data Area

asp=1/1

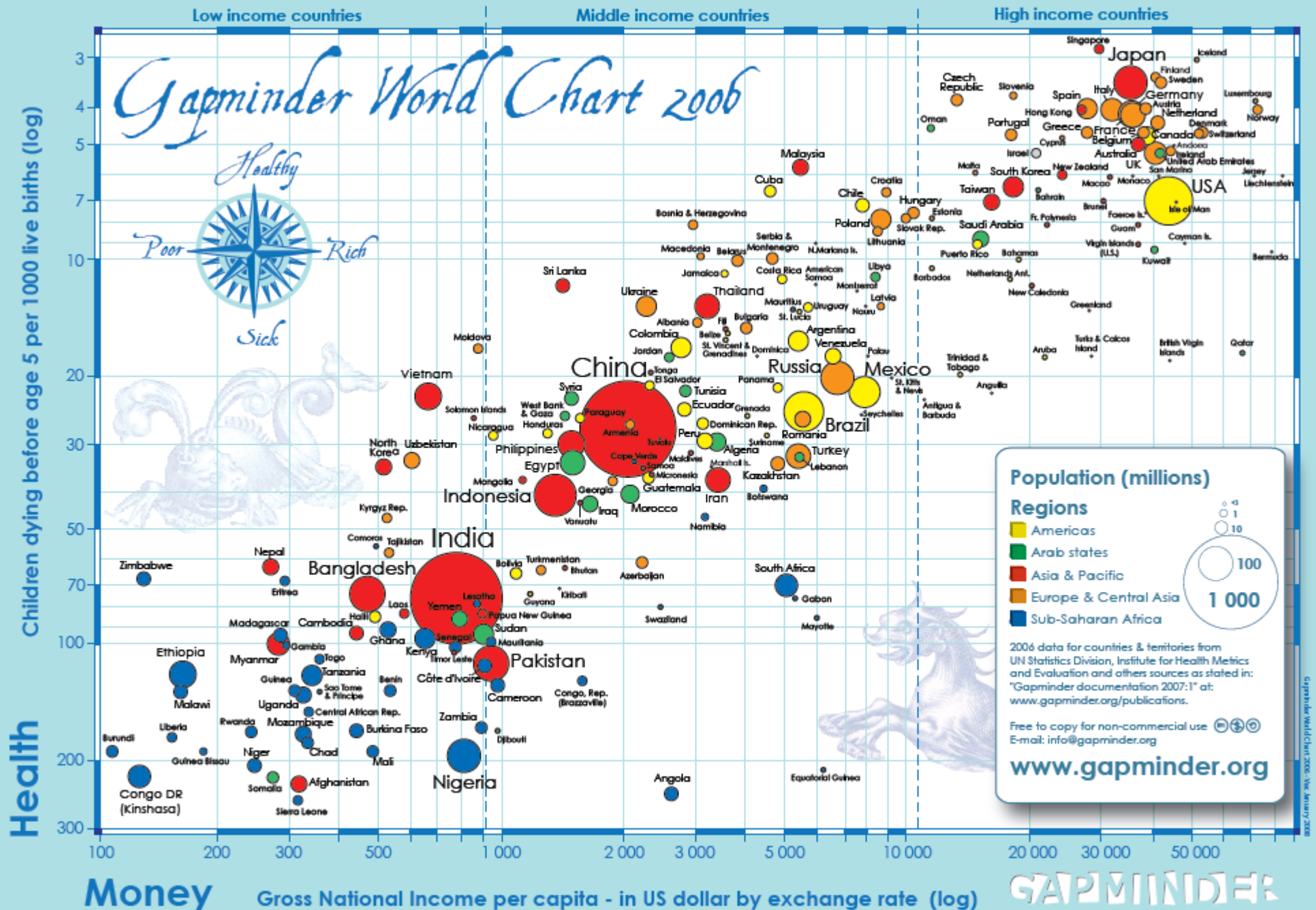


asp=6/1

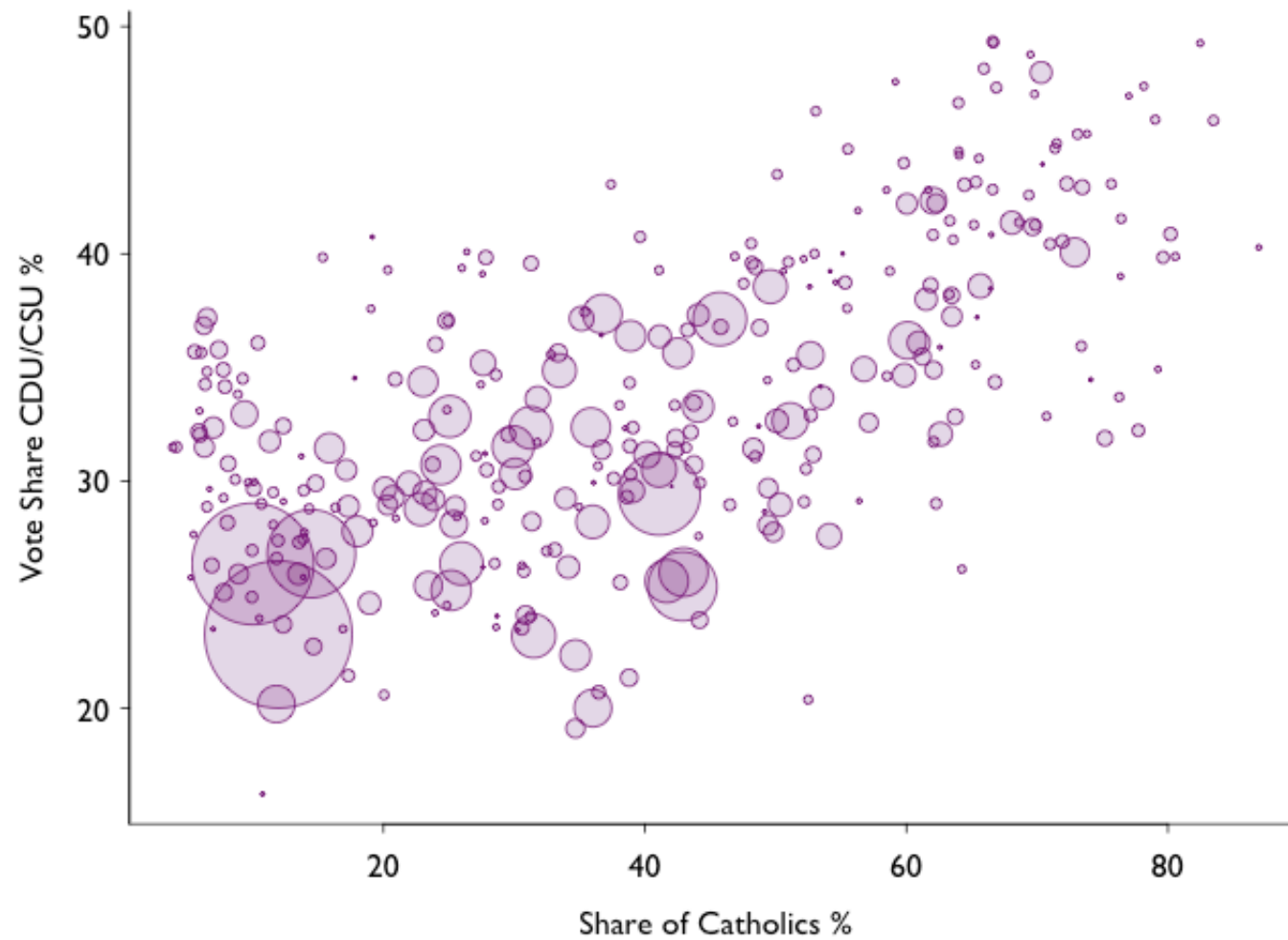


asp=2/5

asp=2.7/1

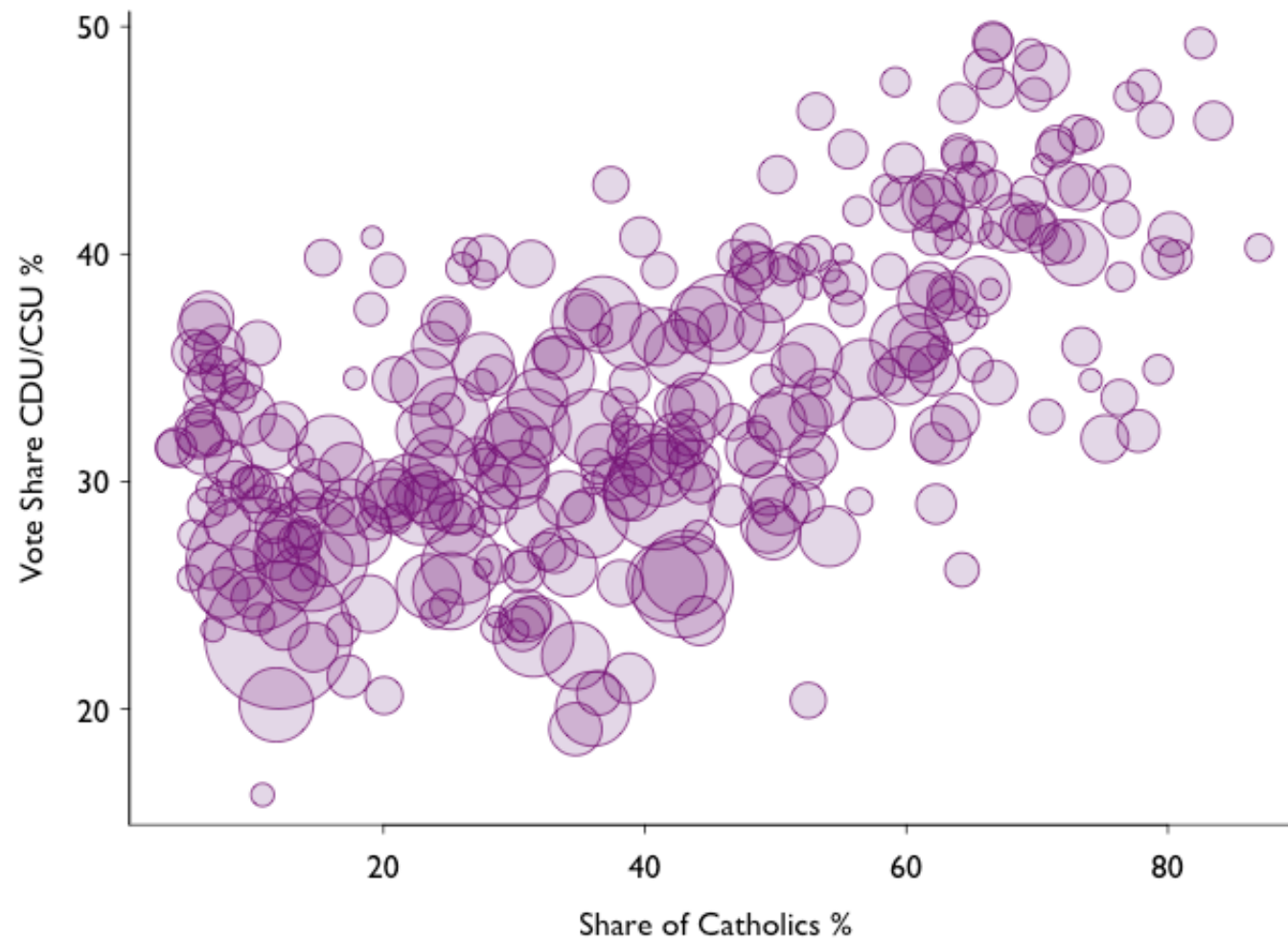


## The Bubble Plot



```
symbols(daten05$kath, daten05$cdu, circles=daten05$n, inches=0.5,  
        bg=rgb(120, 00, 120, 50, max=255), fg=rgb(120, 00, 120,  
        max=255), add=T)
```

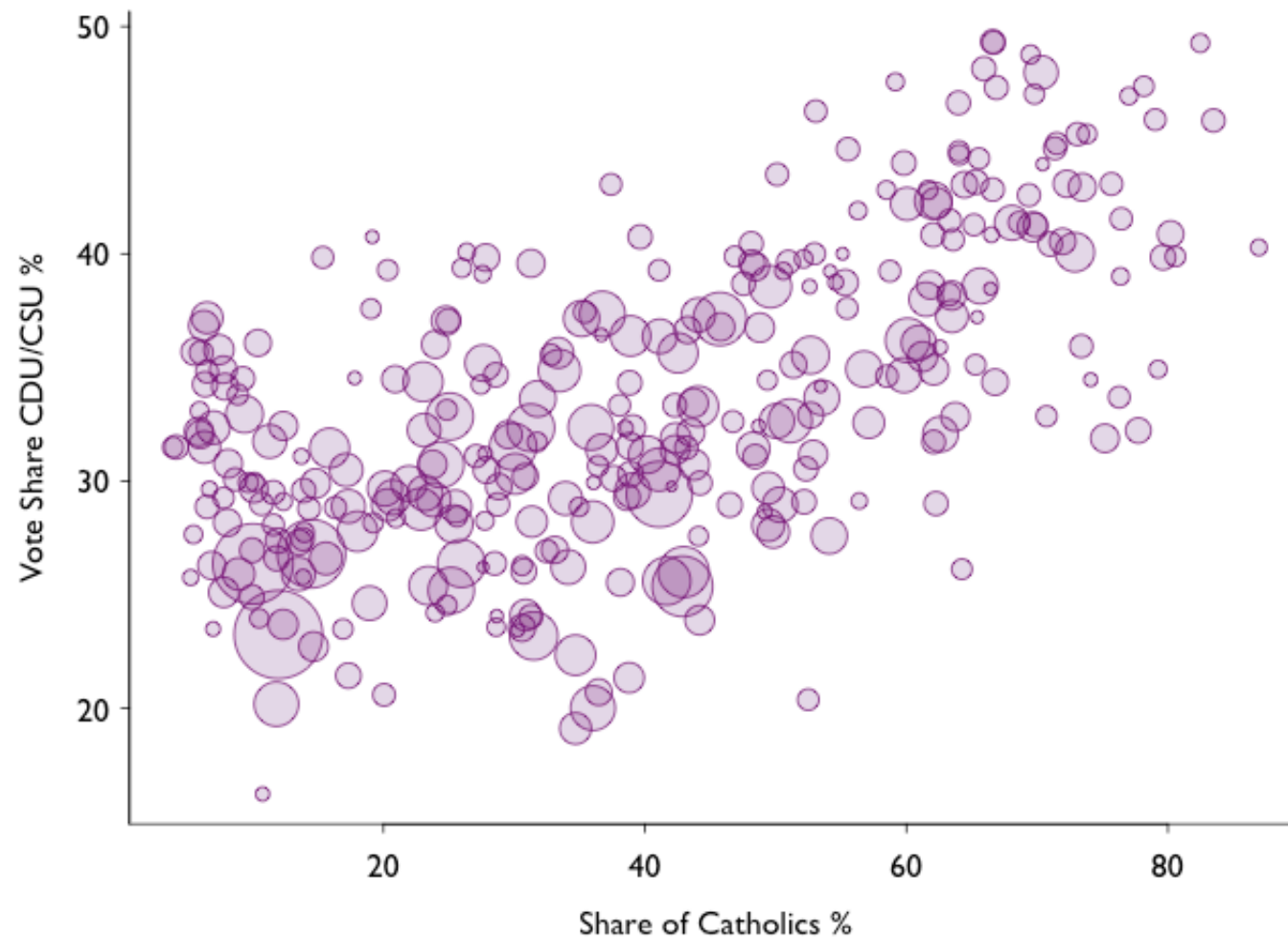
## The Bubble Plot



**ATTENTION:** you need to size the bubbles proportional to area (not radius)!

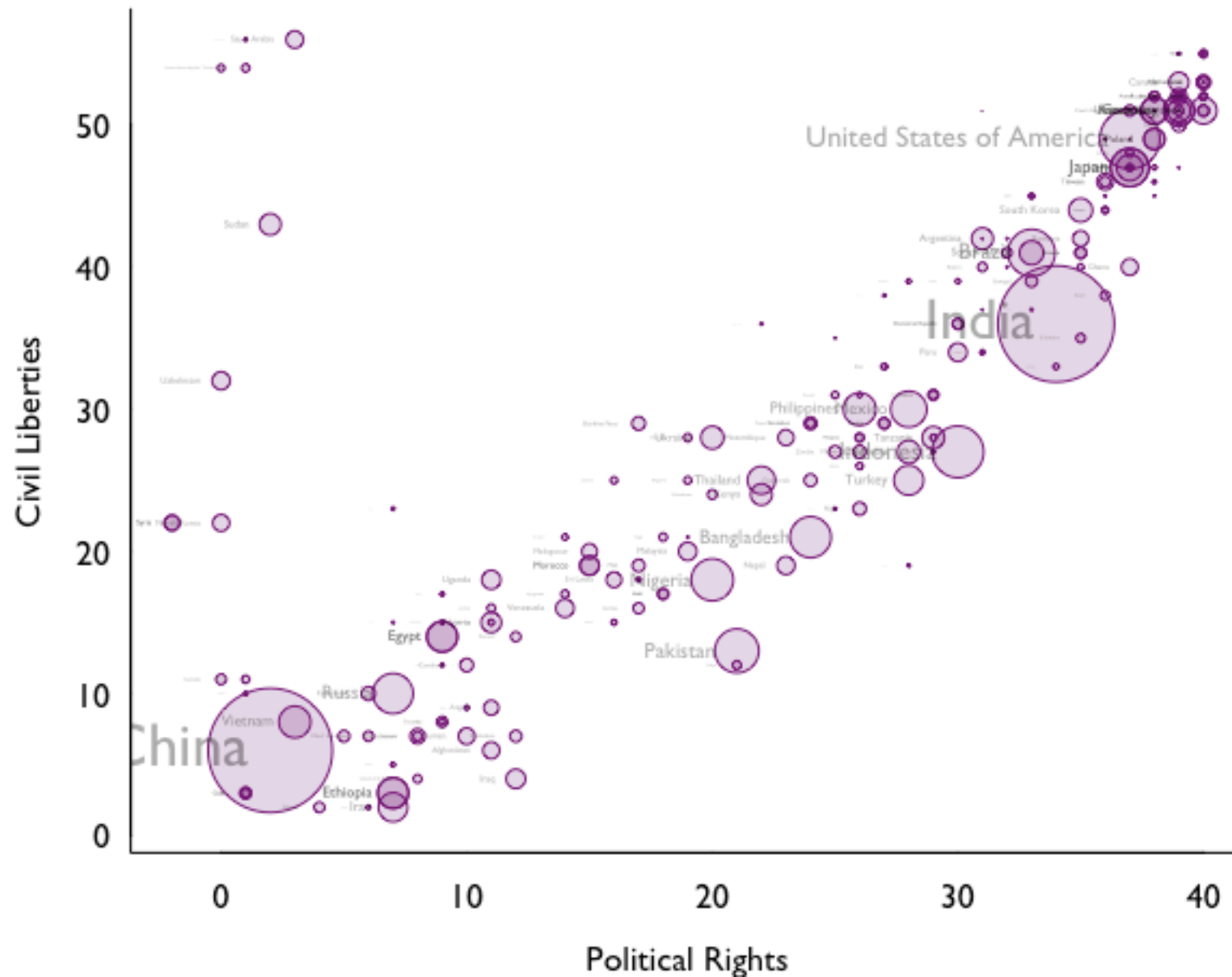
$$Area = \pi r^2 \quad \text{and} \quad r = \sqrt{Area/\pi}$$

## The Bubble Plot



```
symbols(daten05$kath, daten05$cdu, circles=sqrt(daten05$n/pi),  
inches=0.3, bg=rgb(120, 00, 120, 50, max=255), fg=rgb(120, 00, 120,  
max=255), add=T)
```

Adding Text Labels (This looks pretty Rosling-style to me!)



```
text(data$pr_2014, data$cl_2014, data$Country, cex=sqrt((data$pop/pi),  
col=rgb(0, 0, 0, 100, max=255), pos=2)
```